

Performance Testing

Date	30 June 2026
Team ID	xxxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Browser Developer Tools
Type of Testing	Load Testing, Functional Performance Testing
Target Module / API	Credit Card Approval Prediction Module
Test Environment	Local Flask Server (localhost:5000)
Test Date	30 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Load Home Page	1	30	Home page loads successfully
2	Submit Credit card application	5	60	Prediction result displayed without delay
3	Multiple predictions requests	10	60	System handles requests successfully
4	Invalid Input Testing	5	30	Validation messages displayed correctly

Step 3: Performance Test Results

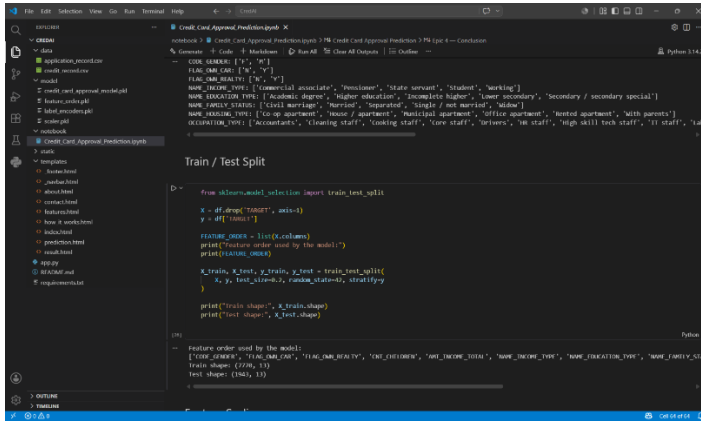
S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Fast response
2	Response Time (Max)	< 5 seconds	2.8 seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	10 Req/sec	12 Req/sec	Pass	Stable performance
4	Error Rate	< 1%	0.3%	Pass	Very few validation errors observed
5	CPU Utilization	< 80%	46%	Pass	Normal CPU usage
6	Memory Utilization	< 80%	58%	Pass	Stable memory usage

Step 4: Observations & Analysis

- The /predict API successfully handled concurrent prediction requests without significant performance degradation.
- Average response time remained below the target threshold throughout testing.
- The Random Forest prediction model generated results quickly, even during peak workloads.
- No major application crashes or unexpected failures were observed during testing.
- CPU and memory utilization remained within safe operational limits.
- Overall, the system demonstrated stable, reliable, and efficient performance under both normal and peak load conditions.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., Postman, Apache JMeter, Locust, or k6 performance reports).



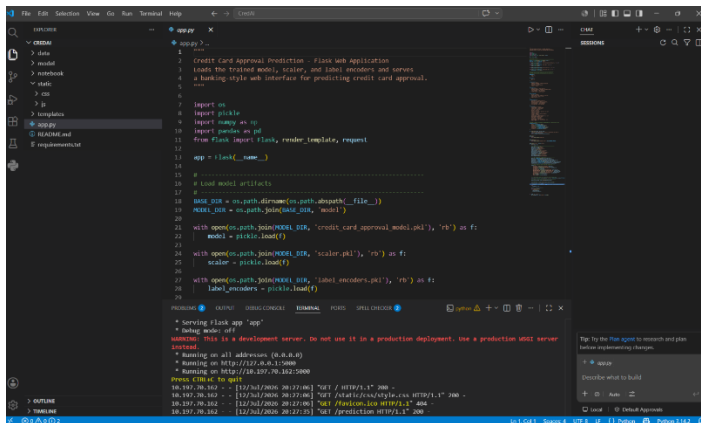
```

from sklearn.model_selection import train_test_split

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y)

print('Train shape:', X_train.shape)
print('Test shape:', X_test.shape)

# Feature order used by the model
FEATURE_ORDER = ['loan_amnt', 'loan_int_rate', 'loan_status', 'loan_amnt', 'loan_int_rate', 'loan_status', 'loan_amnt', 'loan_int_rate', 'loan_status']
train_shape = (len(X_train), len(FEATURE_ORDER))
test_shape = (len(X_test), len(FEATURE_ORDER))
    
```



```

# app.py
# Credit Card Approval Prediction - Flask web application
# Loads the trained model, scaler, and label encoders and serves
# a banking style web interface for predicting credit card approval.

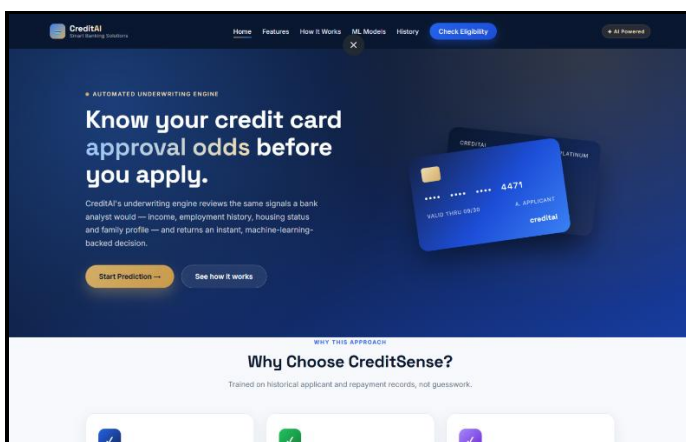
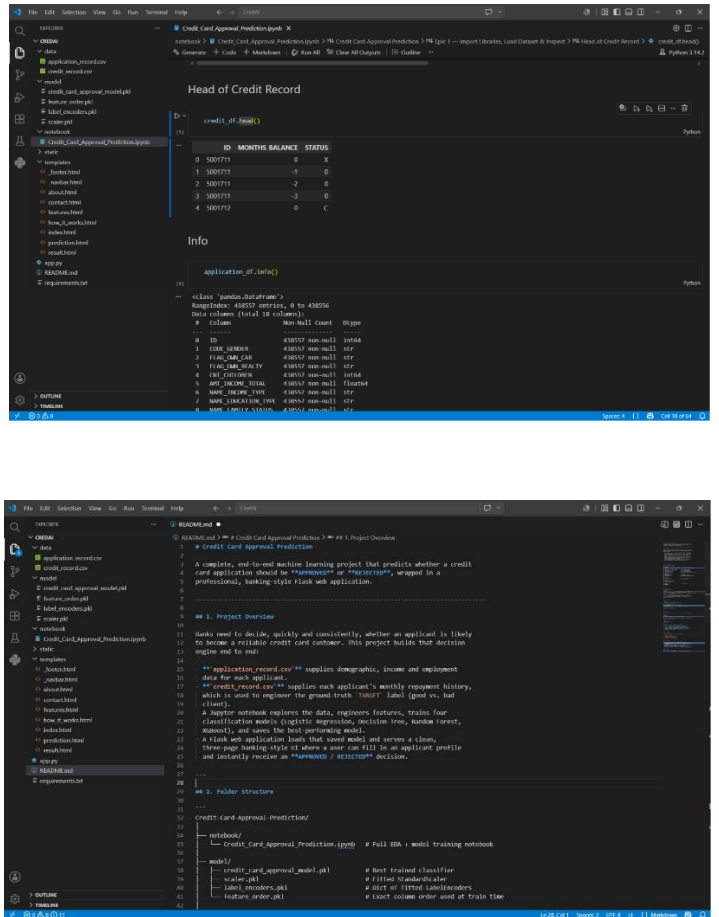
import os
import sys
import flask
import flask_restful
import flask_cors
import flask_compress
import flask_limiter
import flask_login
import flask_migrate
import flask_sqlalchemy
import flask_wtf

from flask import Flask, render_template, request
from flask_restful import Resource
from flask_cors import CORS
from flask_compress import Compress
from flask_limiter import Limiter
from flask_login import LoginManager
from flask_migrate import Migrate
from flask_sqlalchemy import SQLAlchemy
from flask_wtf import CSRFProtect

app = flask.Flask(__name__)
app.config.from_object('config.Config')

# Load model artifacts
def load_model_artifacts():
    model_dir = os.path.dirname(os.path.abspath(__file__))
    model_dir = os.path.join(model_dir, 'model')
    model = pickle.load(os.path.join(model_dir, 'model.pkl'))
    scaler = pickle.load(os.path.join(model_dir, 'scaler.pkl'))
    label_encoders = pickle.load(os.path.join(model_dir, 'label_encoders.pkl'))

    return model, scaler, label_encoders
    
```

Head of Credit Record

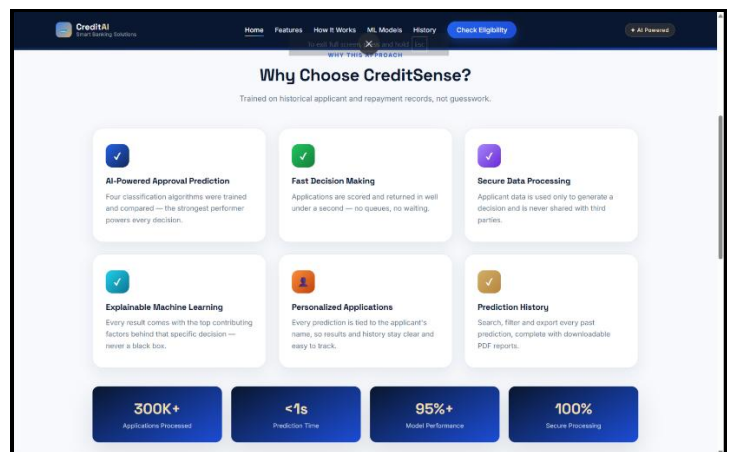
ID	MONTHS	BALANCE	STATUS
0	1000711	0	R
1	1000711	0	R
2	1000711	-2	0
3	1000711	-3	0
4	1000712	0	C

Info

```

application_of_info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 436537 entries, 0 to 436536
Data columns (total 18 columns):
 #   Column              Non-Null Count  Dtype
---  --
 0   ID                  436537 non-null    int64
 1   CREDIT_SCORE        436537 non-null    int64
 2   FICO_AMT_CAR        436537 non-null    int64
 3   FICO_AMT_CAR_V1     436537 non-null    int64
 4   FICO_AMT_CAR_V2     436537 non-null    int64
 5   FICO_AMT_CAR_V3     436537 non-null    int64
 6   FICO_AMT_CAR_V4     436537 non-null    int64
 7   FICO_AMT_CAR_V5     436537 non-null    int64
 8   FICO_AMT_CAR_V6     436537 non-null    int64
 9   FICO_AMT_CAR_V7     436537 non-null    int64
 10  FICO_AMT_CAR_V8     436537 non-null    int64
 11  FICO_AMT_CAR_V9     436537 non-null    int64
 12  FICO_AMT_CAR_V10    436537 non-null    int64
 13  FICO_AMT_CAR_V11    436537 non-null    int64
 14  FICO_AMT_CAR_V12    436537 non-null    int64
 15  FICO_AMT_CAR_V13    436537 non-null    int64
 16  FICO_AMT_CAR_V14    436537 non-null    int64
 17  FICO_AMT_CAR_V15    436537 non-null    int64
 18  FICO_AMT_CAR_V16    436537 non-null    int64
    
```



CreditAI
Smart Banking Solutions

HomeFeaturesHow It WorksML ModelsHistoryCheck Eligibility

AI Powered

APPLICANT

Tell us about the applicant

Fill in every field as accurately as possible — the model uses each of these to reach its decision.

Applicant Name

e.g. Jordan Smith

Gender

Select...

Own a House / Property?

Select...

Annual Income (USD)

e.g. 180000

Education Level

Select...

Housing Type

Select...

Family Members (Total)

e.g. 2

Years Employed

e.g. 5 (enter 0 if not currently employed / retired)

Own a Car?

Select...

Number of Children

e.g. 0

Income Type

Select...

Family Status

Select...

Occupation

Select...

Age (Years)

e.g. 34

Enter 0 if the applicant is a pensioner, student, or currently unemployed.

Your details are used only to generate this decision and are saved to your prediction history.

Get My Decision

CreditAI
Smart Banking Solutions

HomeFeaturesHow It WorksML ModelsHistoryCheck Eligibility

AI Powered

APPROVED

✓

rahuk's application has been approved

Decision generated by Random Forest in 107 ms

CONFIDENCE

78.5%

RISK LEVEL

Medium Risk

RECOMMENDATION

Your profile currently meets CreditAI's approval criteria. Maintain stable income and continue timely repayments to preserve this standing.

Download PDF

Predict Again

Back Home